

SETUL DE INSTRUCȚIUNI AL μ P Z - 80 (III)

1.Obiectul lucrării

Se continuă studiul setului de instrucțiuni al microprocesorului Z-80 cu următoarele grupe: instrucțiunile de salt, instrucțiunile aritmetice pe 16 biți, instrucțiunile de lucru cu acumulatorul și diverse, instrucțiunile pentru apelul subrutinelor, instrucțiunile de revenire din subrutine și, instrucțiunile de restart. Se prezintă aplicații cu acestea.

2.Breviar teoretic

VII. Grupa instrucțiunilor de salt

Instrucțiunile din cadrul acestei grupe acționează asupra registrului PC (Programm Counter), provocând schimbarea conținutului acestuia. Dar, registrul PC adresează memoria în vederea extragerii codului instrucțiunii ce urmează a fi executate de microprocesor. Se forțează astfel execuția următoarei instrucțiuni de la adresă specificată prin constanta pe 16 biți ce se încarcă în PC. Practic, acest lucru provoacă un salt în derularea liniară a unui program și de aici denumirea acestei grupe de instrucțiuni.

Instrucțiunile de salt la microprocesorul Z-80 sunt de două tipuri : **directe** și **indirecte**.

Salturile directe se caracterizează prin faptul ca în PC se depune direct constanta pe 16 biți ce reprezintă adresă de salt.

Salturile indirecte implică transferul de la un alt registru (deci indirect), a adresei de salt, în registrul PC.

μ P Z-80 poate efectua două categorii de instrucțiuni de salt de tip direct:

- **absolute**: saltul se face la o adresă absolută de memorie - număr pe 16 biți;

- **relative** : saltul se face luând ca referință valoarea conținută în PC-ul actual. Se indică, folosind codul complement față de doi, numărul de locații peste care se sare (înainte sau înapoi față de PC-ul curent). Cu instrucțiuni de tip salt relativ se pot scrie programe cu caracter relocabil. Astfel de programe pot fi mutate oriunde în spațiul memoriei și vor putea fi rulate oricând din locul respectiv, obținând aceleași rezultate indiferent de acest loc.

Ambele categorii de instrucțiuni de salt direct prezintă câte două subcategorii:

- **necondiționate**: saltul se efectuează forțat;

Îndrumar de laborator 4

- **condiționate** : saltul se efectuează ținându-se seama de valoarea indicatorilor de condiții.

A. Subgrupa instrucțiunii de salt absolut, necondiționat, direct

JP n16 - (jump)

Simbolic : $PC \leftarrow n16$; Constanta pe 16 biți - n16 - este transferată necondiționat în registrul PC, provocând un salt la respectiva adresă la rularea programului.

Instrucțiune	S Z H O N C	t	L	Cod mașină	Exemplu
JP n16	- - - - -	10	3	C3h n16 _L n16 _H	JP 200h PC←200h

B. Subgrupa instrucțiunilor de salt absolut, condiționat, direct

Prin intermediul acestor instrucțiuni se testează bistabilii din cadrul registrului indicatorilor de condiții, putându-se lua decizii în funcție de valoarea lor logică. Prin intermediul acestor instrucțiuni se pot implementa blocurile logice de tip “ test ” din cadrul organigramelor.

Se pot testa patru din cei șase indicatori: Z, Cy, P/V, S .

Dacă condiția testată nu este îndeplinită, nu se execută salt și procesorul trece la execuția instrucțiunii imediat următoare celei de salt condiționat.

Există și alte instrucțiuni ce permit testarea indicatorilor de condiții.

Generic, instrucțiunile din această grupă (8 instrucțiuni) se pot scrie:

JP cc, n16 - (jump if cc)

$cc \in \{ NZ, Z, NC, C, PO, PE, P, M \}$

c c	codificare (c c c)	SEMNIFICAȚIE
NZ	0 0 0	not zero - se execută instrucțiunea (saltul), dacă indicatorul Z din registrul F este 0 (rezultat nenul) ;
Z	0 0 1	zero - se execută instrucțiunea (saltul), dacă indicatorul Z din registrul F este 1 (rezultat nul) ;
NC	0 1 0	not carry - se execută instrucțiunea (saltul), dacă indicatorul C din registrul F este 0 (rezultatul nu este însoțit de transport) ;
C	0 1 1	carry - se execută instrucțiunea (saltul), dacă indicatorul C din registrul F este 1 (rezultatul este însoțit de transport) ;

Îndrumar de laborator 4

PO	1 0 0	parity odd - se execută instrucțiunea (saltul), dacă indicatorul P din registrul F este 0 (rezultatul obținut este impar - odd -) ;
PE	1 0 1	parity even - se execută instrucțiunea (saltul), dacă indicatorul P din registrul F este 1 (rezultatul obținut este par - even -) ;
P	1 1 0	positive - se execută instrucțiunea (saltul), dacă indicatorul S din registrul F este 0 (rezultatul obținut este pozitiv) ;
M	1 1 1	minus - se execută instrucțiunea (saltul), dacă indicatorul S din registrul F este 1 (rezultatul obținut este negativ) ;

OBSERVAȚIE . Un număr este par dacă suma modulo 2 a biților săi este 0 (sau numărul biților aflați în “ 1 ” logic este un număr par).

Simbolic, aceste instrucțiuni realizează :

$PC \leftarrow n16$ - dacă condiția testată cc este adevărată (se efectuează salt);

sau $PC \leftarrow PC+3$ - dacă condiția testată cc este falsă (se trece la instrucțiunea următoare).

Instrucțiune	S	Z	H	O	N	C	t	L	Cod mașină	Exemplu
JP cc ,n16	-	-	-	-	-	-	10	3	1 1 c c c 0 1 0 n16 _L n16 _H	DEC B JP Z ,200h PC←200h pt. B←0

c c c	condiția testată
0 0 0	NZ (Z=0)
0 0 1	Z (Z=1)
0 1 0	NC (C=0)
0 1 1	C (C=1)
1 0 0	PO (P=0)
1 0 1	PE (P=1)
1 1 0	P (S=0)
1 1 1	M (S=1)

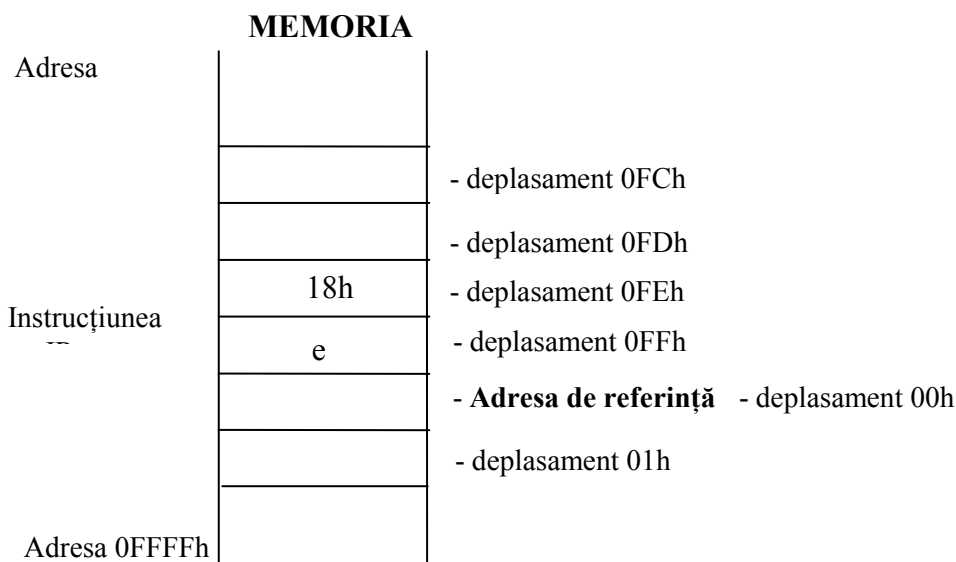
C. Subgrupa instrucțiunii de salt relativ, necondiționat, direct

JR e - jump relative (salt relativ necondiționat) ; e - deplasament exprimat în CC2 . Simbolic, instrucțiunea realizează următorul lucru :

Îndrumar de laborator 4

$PC \leftarrow PC + 2 + e$; Adresa $PC + 2$ este adresă de referință ce se ia în calcul la efectuarea saltului.

Pentru calculul deplasamentului se va avea în vedere următoarea schemă :



Instrucțiune	S	Z	H	O	N	C	t	L	Cod mașină	Exemplu
JR e	-	-	-	-	-	-	12	2	16h e	PC←200h JR ,02h PC←204h

O astfel de instrucțiune efectuează un salt necondiționat la o adresă de memorie aflată la o distanță relativă față de adresa instrucțiunii următoare celei de salt (considerată adresă de referință - vezi figura anterioară). Prin codul instrucțiunii se precizează distanța față de adresa de referință unde se va efectua saltul. Distanța este exprimată printr-un număr reprezentat în cod complement față de 2, denumit deplasament (notat e).

La un deplasament negativ, saltul se face către înapoi (către adresele de început ale memoriei).

Pentru deplasament pozitiv, saltul se face către înainte (către adresele de sfârșit ale memoriei).

Spre exemplu, instrucțiunea JR 00h , va efectua salt necondiționat la instrucțiunea imediat următoare. Instrucțiunea JR 0FEh , va efectua salt chiar pe ea însăși (microprocesorul intrând într-o “ capcană “).

Cum deplasamentul este exprimat doar pe 8 biți, saltul se poate efectua cu maxim 127 locații către înainte, respectiv cu maxim 128 locații către înapoi, față de adresa de referință.

D. Subgrupa instrucțiunilor de salt relativ, condiționat, direct

a) **JR kk, e** - unde kk reprezintă condiția testată, dar spre deosebire de saltul absolut condiționat, kk poate fi doar :

Dacă condiția kk este îndeplinită se efectuează salt, adică:

$PC \leftarrow PC + 2 + e$; Adresa $PC + 2$ este adresă de referință ce se ia în calcul la efectuarea saltului.

Altfel, nu se efectuează salt, adică :

$PC \leftarrow PC + 2$; Se trece la adresa instrucțiunii imediat următoare celei de salt relativ.

Apar următoarele situații:

Instrucțiune	S	Z	H	O	N	C	t	L	Cod mașină	Exemplu
JR NC ,e	-	-	-	-	-	-	12	2	30h ,e	$L \leftarrow 10h$ 1FFh: DEC L 200h: JR Z,0FDh $PC \leftarrow 200h + 02h - 03h$ deci $PC \leftarrow 1FFh$ se repeta secvența de 10h ori
JR C ,e	-	-	-	-	-	-	12	2	38h ,e	
JR NZ ,e	-	-	-	-	-	-	12	2	20h ,e	
JR Z ,e	-	-	-	-	-	-	12	2	28h ,e	

b) **DJNZ e** - decrement and jump relative if not zero (decrementează și salt relativ dacă $Z = 0$)

Simbolic: $B \leftarrow B - 1$

Dacă $B = 0$ se continuă cu următoarea instrucțiune,
iar dacă $B \neq 0$ ($Z = 0$), se face **JR e** (salt relativ cu deplasamentul indicat)

Instrucțiune	S	Z	H	O	N	C	t	L	Cod mașină	Exemplu
DJNZ e	-	-	-	-	-	-	13	2	10h e	

OBSERVAȚIE. Secvența următoare de instrucțiuni:

LD B, n8
et1: DJNZ et1

se constituie într-o temporizare software. Decrementarea din instrucțiunea DJNZ se execută repetat (de n8 ori) până când B = 0. Folosind astfel de procedee, microprocesorul va putea să contorizeze intervale de timp.

E. Subgrupa instrucțiunilor de salt indirect

JP (ss)

Simbolic : $PC \leftarrow ss$;

Conținutul sursei duble ss este transferat necondiționat în registrul PC, provocând un salt la respectiva adresă. Denumirea “indirect” provine din faptul că în prealabil, adresa (pe 16 biți) se încarcă în registrul dublu specificat și apoi (indirect) aceasta ajunge în PC.

$ss \in \{ HL, IX, IY \}$

Apar următoarele situații:

Instrucțiune	S Z H O N C	t	L	Cod mașină	Exemplu
JP (HL)	- - - - -	4	1	E9h	HL ← 100h
JR (IX)	- - - - -	8	2	DDh ,E9h	PC ← 10h
JR (IY)	- - - - -	8	2	FDh ,E9h	JR (HL) PC ← 100h

VIII. Grupa instrucțiunilor aritmetice pe 16 biți

Numărul instrucțiunilor cuprinse în această grupă este mai redus față de cel al instrucțiunilor aritmetice pe 8 biți. Pentru micșorarea numărului de operații pe care le efectuează microprocesorul, este de preferat folosirea acestui tip de instrucțiuni (acolo unde este cazul). Rolul acumulatorului (de la instrucțiunile aritmetice pe 8 biți) este luat de unul din Regiștrii HL, IX sau IY.

A. Subgrupa operațiilor de adunare

a). ADD HL, ss

Simbolic : $HL \leftarrow HL + ss$; ss = sursă dublă $ss \in \{ BC, DE, HL, SP \}$

b). ADC HL, ss - addition with carry (adunare cu transport)

Simbolic : $HL \leftarrow HL + ss + Cy$ $ss \in \{ BC, DE, HL, SP \}$

c) ADD IX, ss

Simbolic : $IX \leftarrow IX + ss$

d) ADD IY, ss

Simbolic : $IY \leftarrow IY + ss$

Îndrumar de laborator 4

$ss \in \{ BC, DE, IX, SP \}$

$ss \in \{ BC, DE, IX, SP \}$

Instrucțiune	S	Z	H	O	N	C	t	L	Cod mașină	Exemplu
ADD HL,ss	-	-	-	-	0	*	11	1	0 0 rp1 rp2 1 0 0 1	HL←46h
ADC HL,ss	*	*	-	*	0	*	15	2	EDh 0 1 rp1 rp2 1 0 1 0	IX←1Fh Cy=0
ADD IX,ss	-	-	-	-	0	*	15	2	DDh 0 0 rp1 rp2 1 0 0 1	ADC HL,IX HL←46h+1Fh+0
ADD IY,ss	-	-	-	-	0	*	15	2	FDh 0 0 rp1 rp2 1 0 0 1	HL←65h

B. Subgrupa operațiilor de scădere

SBC HL, ss - subtract with carry (scădere cu carry)

Simbolic : $HL \leftarrow HL - ss - Cy$ $ss \in \{ BC, DE, HL, SP \}$

Instrucțiune	S	Z	H	O	N	C	t	L	Cod mașină	Exemplu
SBC HL,ss	*	*	-	*	1	*	15	2	EDh 0 1 rp1 rp2 0 0 1 0	HL←65h IY←1Fh Cy=0
										SBC HL,IY HL←46h

OBSERVAȚIE . Nu există alte tipuri de operații de scădere pe 16 biți (fără carry sau cu alți regiștri).

C. Subgrupa operațiilor de incrementare - decrementare pe 16 biți

1) Incrementarea

INC dd

Simbolic : $dd \leftarrow dd + 1$

$dd \in \{ BC, DE, HL, SP, IX, IY \}$

Apar situațiile:

Instrucțiune	S	Z	H	O	N	C	t	L	Cod mașină	Exemplu
INC rp	-	-	-	-	-	-	6	1	0 0 rp1 rp2 0 0 1 1	HL←100h
INC IX	-	-	-	-	-	-	10	2	DDh,23h	INC HL
INC IY	-	-	-	-	-	-	10	2	FDh,23h	HL←101h

$rp \in \{ BC, DE, HL, SP \}$

1) Decrementarea**DEC dd**

Simbolic : $dd \leftarrow dd - 1$ $dd \in \{ BC, DE, HL, SP, IX, IY \}$

Apar situațiile:

Instrucțiune	S	Z	H	O	N	C	t	L	Cod mașină	Exemplu
INC rp	-	-	-	-	-	-	6	1	0 0 rp1 rp2 1 0 1 1	HL ← 100h DEC HL HL ← 0FFh
INC IX	-	-	-	-	-	-	10	2	DDh ,2Bh	
INC IY	-	-	-	-	-	-	10	2	FDh ,2Bh	

rp $\in \{ BC, DE, HL, SP \}$

IX. Grupa instrucțiunilor de lucru cu acumulatorul și registrul indicatorilor de condiții

a) **DAA** - (decimal adjust accumulator) - ajustarea zecimală a acumulatorului.

Transformă codul binar existent în A într-un cod zecimal codat binar (cod BCD), după efectuarea unor operații de adunare sau scădere în care sunt implicați operanzi reprezentați în BCD.

Exemplu : Executând o operație de adunare a două numere, de exemplu 17 (BCD) și 39 (BCD), se obține simplu, în aritmetica zecimală, rezultatul 56. Lucrând cu reprezentări binare :

$$\begin{array}{r} 0001 \ 0111 + \\ \underline{0011 \ 1001} \\ 0101 \ 0000 = 50 \end{array}$$

Rezultatul este incorect. Instrucțiunea DAA ajustează acest rezultat pentru a se obține reprezentarea BCD a sumei :

$$\begin{array}{r} 0101 \ 0000 + \\ \underline{0000 \ 0110} \\ 0101 \ 0110 = 56 \end{array}$$

b) **CPL** - (complement accumulator) - complementarea acumulatorului

Simbolic : $A \leftarrow \overline{A}$ prin această operație se obține codul complement față de 1 (CC1) a datelor existente în acumulator.

c) **NEG** - instrucțiunea transformă numărul binar existent în acumulator în complementul sau față de 2.

Îndrumar de laborator 4

Simbolic : $A \leftarrow 00h - A \quad (A \leftarrow \overline{A} + 01h)$

d) **CCF** - (complement carry flag) - complementarea indicatorul carry din registrul F.

Simbolic : $Cy \leftarrow \overline{C_y}$

e) **SCF** - (set carry flag) - pune în “ 1 ” logic (setează) indicatorul carry.

Simbolic : $Cy \leftarrow 1$

Instrucțiune	S Z H O N C	t	L	Cod mașină	Exemplu
DAA	* * * * - *	4	1	27h	----- -
CPL	- - 1 - 1 -	4	1	2Fh	$A \leftarrow 00h$ CPL $A \leftarrow 0FFh$
NEG	* * * * 1 *	8	2	EDh 44h	$A \leftarrow 01h$ NEG $A \leftarrow 0FFh$
CCF	- - - - 0 *	4	1	3Fh	$Cy=0$ CCF $Cy=1$
SCF	- - 0 - 0 1	4	1	37h	

OBSERVAȚIE . Carry este singurul indicator de condiții asupra căruia se poate acționa în mod direct, prin instrucțiuni (cele anterioare). Nu există instrucțiune pentru resetarea lui Cy; pentru aceasta se face SCF urmat de CCF.

X. Grupa instrucțiunilor diverse

a) **NOP** - (no operation) - nici o operație, procesorul trece la instrucțiunea următoare.

b) **HALT** - oprirea procesorului; din starea HALT nu se poate ieși decât prin resetarea microprocesorului sau prin primirea unei cereri de întrerupere recunoscută de acesta.

c) **DI** -(disable interrupt) - dezactivarea mecanismului de întreruperi mascabile (IFF1 = 0)

d) **EI** - (enable interrupt) - activarea mecanismului de întreruperi mascabile (IFF1 = 1)

e) **IM 0** -(interrupt mode 0)- trecere microprocesorul în modul 0 de tratare a întreruperilor mascabile.

f) **IM 1** - (interrupt mode 1) - provoacă trecerea microprocesorului în modul 1 de tratare a întreruperilor mascabile.

g) **IM 2** - (interrupt mode 2) - provoacă trecerea microprocesorului în modul 2 de tratare a întreruperilor mascabile.

Îndrumar de laborator 4

Instrucțiune	S Z H O N C	t	L	Cod mașină	Exemplu
NOP	- - - - -	4	1	00h	-----
HALT	- - - - -	4	1	76h	A←00h CPL A←0FFh
DI	- - - - -	4	1	F3h	A←01h NEG A←0FFh
EI	- - - - -	4	1	FBh	Cy=0 CCF Cy=1
IM 0	- - - - -	8	2	EDh 46h	-----
IM 1	- - - - -	8	2	EDh 56h	-----
IM 2	- - - - -	8	2	EDh 5Eh	-----

OBSERVAȚIE. Trecerea într-un mod de tratare a întreruperilor mascabile nu realizează și validarea acestui mecanism de întreruperi. În acest scop este necesară rularea unei instrucțiuni de tip EI.

XI. Grupa instrucțiunilor pentru apelul subrutinelor

În derularea programelor există situații când un anumit grup de instrucțiuni se execută repetat.

Pentru a economisi memoria calculatorului, precum și pentru a ușura munca programatorului, zona care se repetă se declară subrutină; ea se scrie o singură dată în derularea programului și se apelează de câte ori este nevoie. Respectiva subrutină se identifică printr-o etichetă a primei instrucțiuni sau prin adresa acesteia. Orice subrutina trebuie să aibă la sfârșitul său logic o instrucțiune de revenire în programul din care a fost apelată, de tipul RETURN. Sunt două tipuri de instrucțiuni pentru apelul subrutinelor :

A. Subgrupa instrucțiunii pentru apelul necondiționat al subrutinelor

CALL n16 - apel necondiționat al subrutinei aflată la adresa specificată prin constanta pe 16 biți n16.

Simbolic: (SP - 1) ⇐ PC_H
 (SP - 2) ⇐ PC_L
 SP ⇐ SP - 2
 PC ⇐ n16

La apelarea unei subrutine se salvează în stivă (folosind SP-ul) adresa instrucțiunii imediat următoare celei la care a intervenit apelul subrutinei (adresă conținută în PC). SP-ul se reactualizează, indicând adresa ultimei depuneri în stivă, iar în PC se încarcă constanta n16 ce specifică adresa subrutinei.

Îndrumar de laborator 4

Instrucțiune	S Z H O N C	t	L	Cod mașină	Exemplu
CALL n16	- - - - -	17	3	CDh n16 _L n16 _H	CALL 1234h CDh 34h 12h

B. Subgrupa instrucțiunilor pentru apelul condiționat al subrutinelor

Instrucțiunile din această subgrupă testează indicatori din registrul F, luând decizia de apelare a subrutinei doar dacă condiția testată este îndeplinită. În caz contrar, se trece la instrucțiunea imediat următoare apelului subrutinei. Sunt 8 instrucțiuni de apel condiționat al subrutinelor (ce testează doar 4 indicatori de condiții), desemnate prin mnemonica generică:

CALL cc, n16 - apelul condiționat al subrutinei aflată la adresă n16, în cazul condiției adevărate cc.

cc ∈ {NZ, Z, NC, C, PO, PE, P, M}

Simbolic: Dacă condiția testată “cc” este îndeplinită se desfășoară aceleași acțiuni ca și la instrucțiunea CALL n16, în caz contrar trecându-se la instrucțiunea imediat următoare apelului subrutinei.

Instrucțiune	S Z H O N C	t	L	Cod mașină	Exemplu
CALL cc ,n16	- - - - -	17	3	1 1 c c c 1 0 0 n16 _L n16 _H	DEC B CALL NZ ,1234h Daca B←0 salt la Adresa 1234h

Codarea condiției testate “ c c c ” se menține ca și la instrucțiunile de tip salt condiționat.

XII. Grupa instrucțiunilor pentru revenirea din subrutine

Orice subrutină trebuie să se termine din punct de vedere logic cu o instrucțiune specială pentru revenirea în programul principal. De remarcat faptul că nu întotdeauna sfârșitul fizic al unei subrutine (ultima instrucțiune a acesteia) coincide cu sfârșitul său logic. Revenirea din subrutină permite refacerea contextului anterior de lucru (reîntoarcerea în programul în care a survenit apelul subrutinei). Există mai multe tipuri de instrucțiuni de revenire:

A. Subgrupa instrucțiunii pentru revenirea necondiționată din subrutină**RET** - (return) - revenire necondiționată din subrutină

Simbolic: $PC_L \leftarrow (SP)$
 $PC_H \leftarrow (SP + 1)$
 $SP \leftarrow SP + 2$

Se reface vechiul PC din momentul apelului subrutinei (în vederea reluării programului ce a apelat subrutina) , iar SP-ul este reactualizat.

Instrucțiune	S	Z	H	O	N	C	t	L	Cod mașină	Exemplu
RET	-	-	-	-	-	-	10	1	C9h	-----

B. Subgrupa instrucțiunilor pentru revenirea condiționată din subrutină

Instrucțiunile din această subgrupă testează indicatori din registrul F, luând decizia de revenire din subrutină doar dacă condiția testată este îndeplinită. În caz contrar, se trece la instrucțiunea imediat următoare, rămânându-se pe mai departe în subrutină.

Sunt 8 instrucțiuni de revenire condiționată din subrutine (ce testează doar 4 indicatori de condiții), desemnate prin mnemonica generică:

RET cc - revenire condiționată din subrutină, dacă condiția testată cc este îndeplinită.

$cc \in \{NZ, Z, NC, C, PO, PE, P, M\}$

Simbolic : Dacă condiția testată “cc” este îndeplinită se desfășoară aceleași acțiuni ca și la instrucțiunea RET, în caz contrar trecându-se la instrucțiunea imediat următoare celei de revenire din subrutină.

Instrucțiune	S	Z	H	O	N	C	t	L	Cod mașină	Exemplu
RET	-	-	-	-	-	-	10	1	11ccc000	-----

Condițiile testate “cc” , precum și codarea binară a acestora “ccc” se mențin ca și la instrucțiunile de tip salt condiționat și apel condiționat de subrutină.

Exemple : RET NZ - return if not zero - cod 0C0h
 RET PE - return if parity even - cod 0E8h

OBSERVAȚIE . Instrucțiunile de tip salt condiționat, apel condiționat de subrutină precum și cele de revenire condiționată din subrutină formează grupa instrucțiunilor prin care se testează indicatorii de condiții. Prin ele se pot implementa în cadrul organigramelor, blocurile logice de decizie.

C. Subgrupa instrucțiunii pentru revenirea din subrutină de tratare a unei întreruperi mascabile

RETI - (return from interrupt) - revenire din subrutină ce tratează întreruperea mascabilă.

Simbolic: Are același efect ca și instrucțiunea **RET**, în plus rearmează perifericul ce a solicitat întrerupere procesorului, în vederea generării unei noi întreruperi în viitor. Se reamintește faptul că într-o astfel de situație și procesorul trebuie rearmat pentru acceptarea unei noi cereri de întrerupere mascabile, prin rularea unei instrucțiuni **EI** (enable interrupt).

Instrucțiune	S	Z	H	O	N	C	t	L	Cod mașină	Exemplu
RETI	-	-	-	-	-	-	14	2	EDh 4Dh	-----

D. Subgrupa instrucțiunii pentru revenirea din subrutină de tratare a unei întreruperi nemascabile

RETN - (return from non-maskable interrupt) - revenire din subrutina de tratare a întreruperii nemascabile.

Se reamintește faptul că la începutul tratării unei întreruperi nemascabile, starea bistabilului IFF1 este copiată în bistabilul IFF2, iar IFF1 este adus în starea "0". Astfel, pe timpul tratării unei întreruperi nemascabile, nu sunt acceptate cererile de întrerupere mascabile.

Simbolic: Este similară cu **RET**, în plus starea bistabilului IFF2 se copiază din bistabilul IFF1. Se revine astfel la starea anterioară a mecanismului de întreruperi mascabile.

Instrucțiune	S	Z	H	O	N	C	t	L	Cod mașină	Exemplu
RETN	-	-	-	-	-	-	14	2	EDh 45h	-----

Reguli de lucru cu subrutinele în cadrul programelor

1. În orice program care utilizează subrutine va trebui să se inițializeze în prealabil baza stivei prin încărcarea registrului SP cu o constantă pe 16 biți ce reprezintă adresa de baza a stivei. Stiva trebuie creată într-o zonă de memorie RAM.

2. Începutul subrutinei trebuie individualizat fie printr-o etichetă, fie prin adresa de memorie de la care este plasat corpul subrutinei. În programele sursă, subrutina va fi apelată prin eticheta ce o individualizează, iar în programele obiect prin adresă pe 16 biți.

Îndrumar de laborator 4

3. Orice subrutină trebuie să aibă sfârșitul logic marcat printr-o instrucțiune RETURN (fie condiționată, fie necondiționată). Nu întotdeauna sfârșitul logic coincide cu sfârșitul fizic (ultima instrucțiune).

4. Dacă subrutina, prin execuția ei, afectează regiștri procesorului care mai târziu vor fi folosiți după revenirea în programul principal, atunci primele instrucțiuni din corpul subrutinei vor trebui să salveze în stivă conținutul acestor regiștri. Înainte de revenirea în programul principal, regiștri salvați vor fi restaurați din stivă, în ordinea inversă depunerii lor (păstrând mecanismul de adresare a stivei de tip LIFO). Vor fi folosite în acest scop instrucțiuni de tip PUSH și POP.

Exemplu :

SUBR :	PUSH BC PUSH DE PUSH HL PUSH AF . . . POP AF POP HL POP DE POP BC RET	}	Depunerea în stivă
	.	}	Codul propriu-zis al subrutinei
	POP AF POP HL POP DE POP BC RET	}	Restaurarea regiștrilor în ordinea inversă depunerii în stivă

XIII. Grupa instrucțiunilor de RESTART

Aceste instrucțiuni sunt în fond tot instrucțiuni pentru apelul necondiționat al subrutinelor, dar care sunt scrise la adrese fixe, rezervate de memorie, în pagina 0 a acesteia. Pagina 0 a memoriei este reprezentată de acele locații de memorie pentru care ocms-ul adresei este 00h (256 locații cuprinse între 0000h și 00FFh). Sunt 8 instrucțiuni de tip RESTART având mnemonica generică:

RST i $i \in \{ 0 \div 7 \}$.

Simbolic:

$(SP - 1)$	$\Leftarrow PC_H$
$(SP - 2)$	$\Leftarrow PC_L$
SP	$\Leftarrow SP - 2$
PC _H	$\Leftarrow 00h$
PC _L	$\Leftarrow n_i$

unde $n_i \in \{ 00h, 08h, 10h, 18h, 20h, 28h, 30h, 38h \}$, (funcție de valoarea lui i).

Instrucțiune	S	Z	H	O	N	C	t	L	Cod mașină	Exemplu
RST i	-	-	-	-	-	-	11	1	11 i i i 1 1 1	-----

i i i este funcție de i (din codul instrucțiunii RST i)

i	i i i
0	0 0 0
1	0 0 1
2	0 1 0
3	0 1 1
4	1 0 0
5	1 0 1
6	1 1 0
7	1 1 1

Spre exemplu, instrucțiunea RST 7 este echivalentă cu instrucțiunea CALL 0038h sau instrucțiunea RST 1 este echivalentă cu CALL 0008h.

Deosebirea între instrucțiunile CALL n16 și RST i, constă în numărul de octeți prin care se reprezintă în memorie (CALL - 3 octeți, respectiv RST - 1 octet). Pe de altă parte, CALL poate apela o subrutină aflată oriunde în memorie, în schimb RST apelează doar subrutine scrise în pagina 0 a memoriei, la adrese rezervate.

3. PROBLEME REZOLVATE

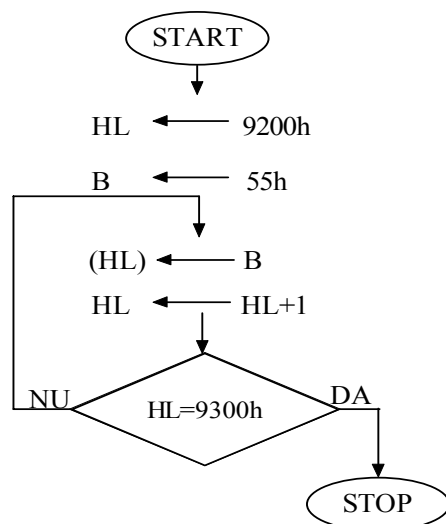
1. Realizați un program în care locațiile de adrese ale zonei de memorie cuprinse între adresele 9200h și 9300h vor fi inițializate cu constanta 55h.

Soluție

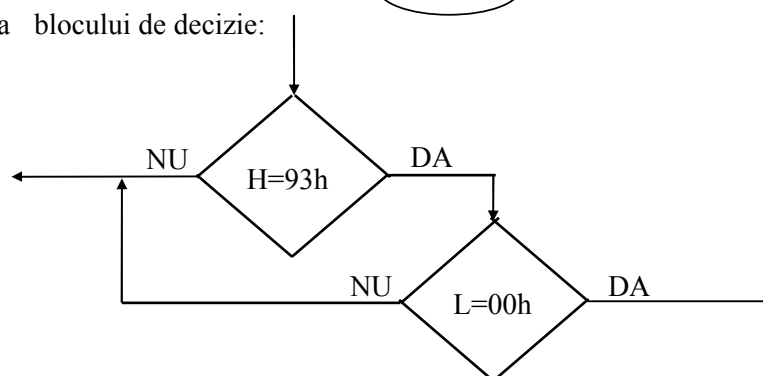
Vom folosi un registru dublu (16 biți), pentru adresarea locației de memorie - alegem registrul HL. Vom folosi un registru simplu (8 biți), pentru păstrarea constantei - alegem registrul B.

Modalitatea de rezolvare implică folosirea acestor regiștri pe post de regiștri indicatori (pointeri). Inițial, în ei trebuie încărcate valorile inițiale (operația de inițializare). Algoritmul de rezolvare trebuie conceput de așa natură încât eficiența programării să fie cât mai mare. Se impune realizarea unei bucle de program, în corpul buclei realizându-se operațiile de transfer a datei în locația de memorie și de incrementare a adresei. ieșirea din buclă se va realiza după atingerea adresei de sfârșit a zonei de memorie - 300h. Organigrama generală de rezolvare a acestei probleme este prezentată în cele ce urmează.

Îndrumar de laborator 4

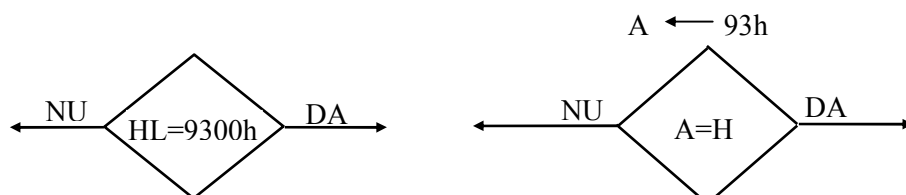


Explicitarea blocului de decizie:



Posibilități de realizare a acestui test (echivalente situației date) :

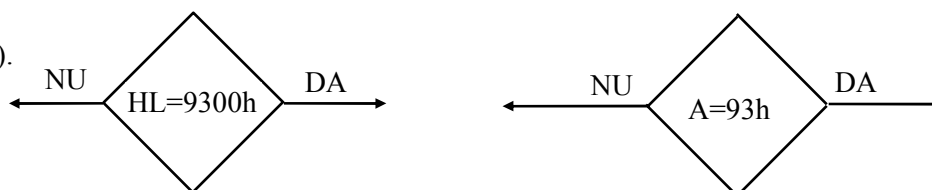
1)



Se compară H cu A

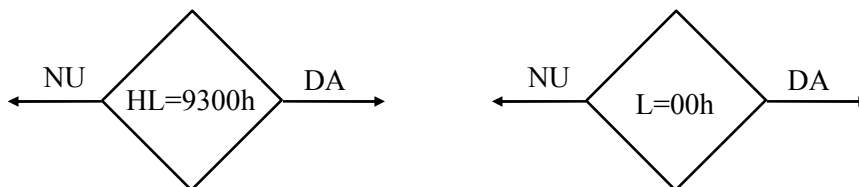
A ← H

2).



Se compară A cu o constantă pe 8 biți

3)



OBSERVAȚIE. În situația 3), testul $L = 00h$ este posibil să se realizeze doar prin analizarea indicatorului Z. Astfel, dacă se va utiliza operația de incrementare a registrului L (incrementare pe 8 biți), în momentul în care L devine 00h (la atingerea adresei 300h), indicatorul Z trece în “1” logic.

În această versiune (3), programul sursă și echivalentul său obiect sunt:

	ORG 8100h	
	LD HL,9200h	8100h: 21h, 00h, 92h
	LD B,55h	8103h: 06h, 55h
et1:	LD (HL),B	8105h: 70h
	INC L	8106h: 2Ch
	JP NZ , et1	8107h: C2h, 05h, 01h
	HALT	810Ah: 76h
	END	

Rezolvare în cazul primei variante:

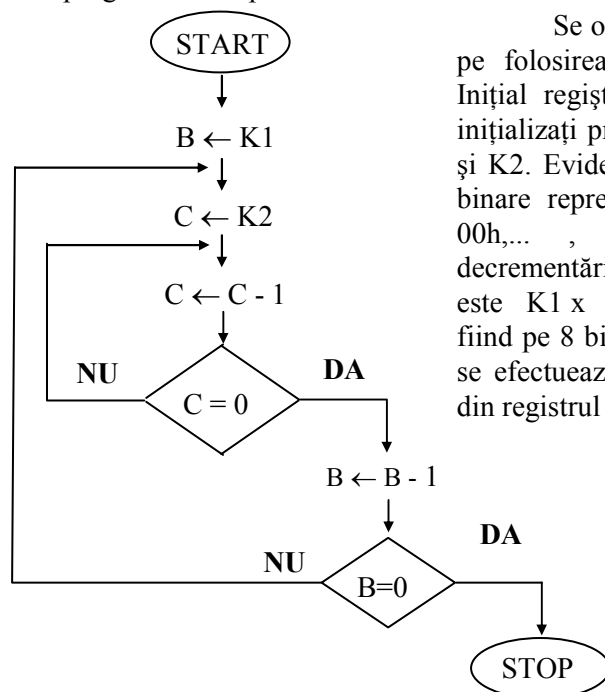
	ORG 8100h	
	LD HL,9200h	8100h: 21h, 00h, 92h
	LD B,55h	8103h: 06h, 55h
et1:	LD (HL),B	8105h: 70h
	INC HL	8106h: 23h
	LD A,03h	8107h: 3Eh, 03h
	CP H	8109h: BCh
	JPNZ et1	810Ah: C2h, 05h, 01h
	LD A,L	810Dh: 7Dh
	CP 00h	810Eh: FEh, 00h
	JPNZ et1	8110h: C2h, 05h, 01h
	HALT	8113h: 76h
	END	

- urmăriți programul pas cu pas și observați care sunt valorile registrului PC pe parcursul rulării lui;
- modificați programul pentru a inițializa locațiile zonei de memorie cu constanta 30 dar în ordine descrescătoare a adreselor;
- ce modificări aduceți programului dacă se pune condiția de sfârșit de șir înaintea instrucțiunii de inițializare locațiilor de memorie cu constanta 55h?

2. Realizați un program de măsurare a timpului (temporizare) software

Soluție

O soluție a acestei probleme se bazează pe efectuarea unor instrucțiuni a căror durată de execuție este riguros controlată. După cum se cunoaște (vezi cursul) o instrucțiune este formată din mai mulți cicli mașină. La rândul său, un ciclu mașină este format din mai multe perioade elementare (număr specific pentru fiecare instrucțiune), practic perioade ale semnalului de ceas care se aplică microprocesorului. Cum semnalul de ceas este accesibil utilizatorului, rezultă că durata de execuție a unei instrucțiuni este cunoscută. O posibilă organigramă a unui program de temporizare este următoarea:



Se observă că programul se bazează pe folosirea operațiilor de decrementare. Inițial regiștrii implicați - B și C - sunt inițializați prin încărcare cu constantele K1 și K2. Evident, acestea pot fi doar numere binare reprezentate pe un octet, în gama 00h,..., 0FFh. Numărul total de decrementări ce sunt realizate de algoritm este $K1 \times K2$. Operațiile de decrementare fiind pe 8 biți, testele pentru ajungerea la 0 se efectuează direct asupra indicatorului Z din registrul indicatorilor de condiții.

Implementarea organigramei generează următorul program sursă și echivalentul său obiect:

	ORG 8200h	
TMP:	LD B,K1	8200h: 06h, K1h * 7T
ET2 :	LD C,K2	8202h: 0Eh, K2h * 7T
ET1 :	DEC C	8204h: 0Dh * 4T
	JP NZ,ET1	8205h: C2h, 04h, 02h * 10T
	DEC B	8208h: 05h * 4T
	JP NZ,ET2	8209h: C2h, 02h, 02h * 10T
	HALT	820Ch: 76h * 4T
	END	

Îndrumar de laborator 4

În partea dreaptă sunt prezentate numărul perioadelor elementare de ceas (T) care se aplică microprocesorului și pe durata cărora se execută respectivele instrucțiuni. Ținând seama de cele două constante $K1$ și $K2$ și de formatul organigramei, rezultă următorul calcul al timpului de execuție al temporizării (t):

$$t = 7T + [7T + (4T + 10T) * K2 + 4T + 10T] * K1 + 4T = 11T + (21T + 14T * K2) * K1$$

Determinarea lui T este simplă. Se pleacă de la frecvența semnalului de ceas ce se aplică microprocesorului (de obicei chiar frecvența cristalului de cuarț) și se inversează respectiva valoare.

OBSERVAȚIE . Problema se pune de obicei invers . Se dau t , T și se cere să se determine $K1$ și $K2$. În acest caz se alege $K1$ în intervalul 00h, ... , 0FFh și se calculează $K2$ folosind formula de mai sus (în condițiile acestei organigrame).

Exemplu : Se dă $f = 2,5$ Mhz (frecvența semnalului de ceas ce se aplică microprocesorului). Se cer valorile lui $K1$ și $K2$ pentru care $t=0,2$ secunde (în condițiile problemei anterioare).

Rezolvare

Din $f = 2,5$ Mhz $\Rightarrow T = 400$ ns . Alegem spre exemplu, $K1 = 250$ (0FAh). Va rezulta din formula de mai sus :

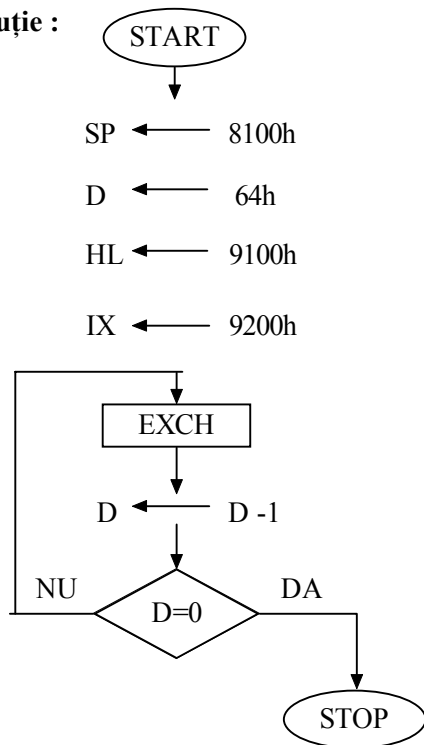
$$K2 = [(t - 11T) / K1 - 21T] / 14 T$$

După calcule se obține $K2 = 141,3$; alegem rotunjit la întreg $K2 = 141$, adică 8Dh. Se poate recalcula valoarea temporizării rezultate cu cele două valori pentru $K1$ și $K2$. Dacă eroarea la care s-a ajuns nu este satisfăcătoare, calculul se poate relua cu o nouă valoare pentru $K1$, obținând un nou $K2$.

3. Se dau două șiruri de câte 100 de octeți plasați de la adresele 9100h respectiv 9200h. Să se realizeze un program în care se schimbă între ele elementele celor două șiruri. Se dorește ca operația de schimb a celor două locații de memorie să fie făcută într-o subrutină.

Îndrumar de laborator 4

Soluție :



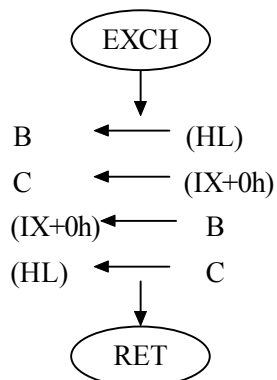
```

ORG 8200h
200h : LD SP, 8100h
201h : LD D, 64h
203h : LD HL, 9100h
206h : LD IX, 9200h
20Ah : ET1: CALL EXCH
20Dh : DEC D
210h : JP NZ ET1
211h : HALT
END
    
```

Inițial se folosesc regiștri dubli HL si IX pentru adresarea indirectă a elementelor celor două șiruri, valorile acestora salvându-se în regiștri B și C. În subrutină se face schimbul propriu-zis. Registrul D contorizează numărul de schimburi acesta fiind testat de fiecare dată cu ajutorul unui bloc de decizie astfel se asigură oprirea programului după terminarea schimburilor.

Este necesara transformarea 100 =64h.

Când se execută instrucțiunea CALL EXCH în stiva aflată la 8100h se salvează adresa instrucțiunii imediat următoare (20Dh) și anume (80FF) : 0Dh iar la (80FE) : 02h.



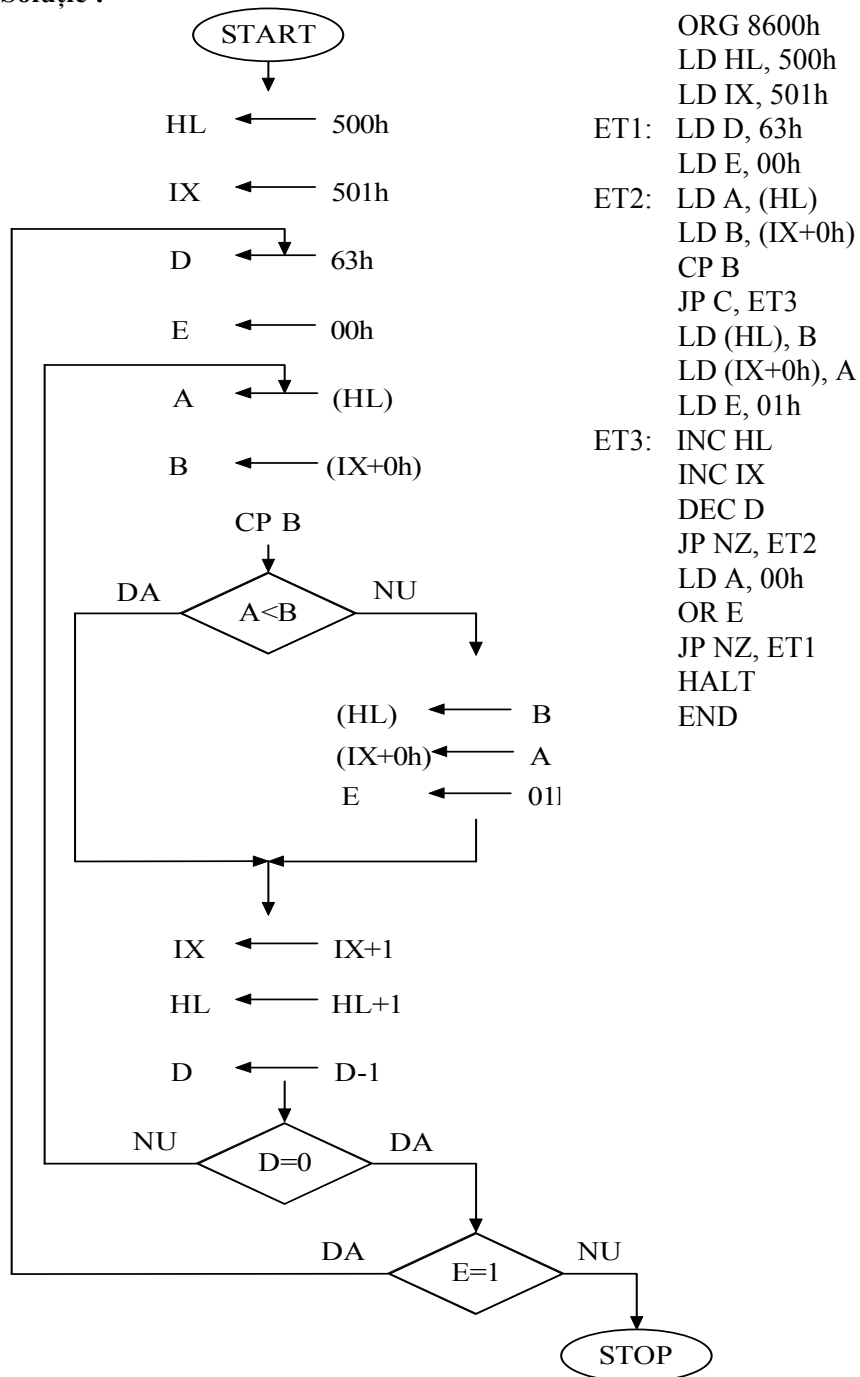
```

EXCH:
ORG 8300h
LD B, (HL)
LD C, (IX+0h)
LD (IX+0h), B
LD (HL), C
INC HL
INC IX
RET
    
```

Îndrumar de laborator 4

4. Să se realizeze un program în care să se ordoneze crescător un șir de 100 de octeți situat începând de la adresa 500h folosind metoda bulelor de aer .

Soluție :



Îndrumar de laborator 4

Metoda bulelor de aer presupune următoarele:

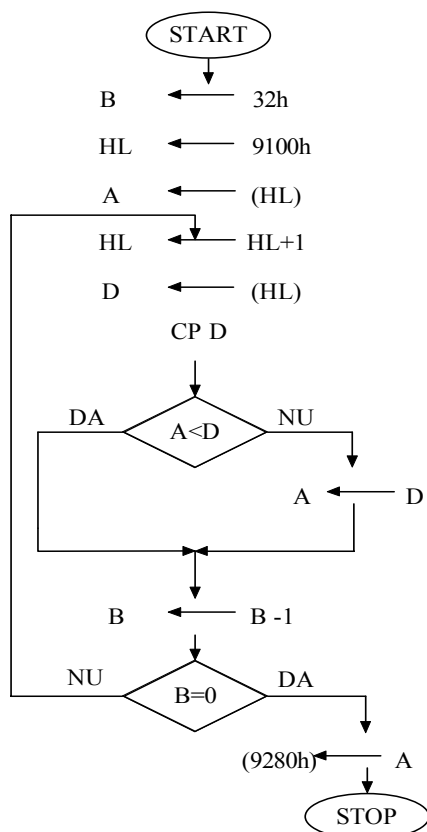
Se compară primele 2 elemente ale șirului și în cazul nostru cel mai mare trece în poziția a doua apoi se continuă comparația cu elementele 2 și 3 ; 3 și 4 până la sfârșitul șirului. În cadrul procesului de comparație la fiecare schimbare ce are loc se pune valoarea 1 în registrul E ce este folosit pe post de semafor. Dacă la testarea acestui semafor se găsește în el valoarea 0 (deci în urma comparării nu s-a făcut nici un schimb) înseamnă că șirul a fost ordonat și programul se oprește. În program se folosește un contor care ține numărul de elemente ale șirului rămase de comparat [reg. C].

6. Să se realizeze un program în care să se determine minimul dintr-un sir de 50 de octeți situat in memorie începând cu adresa 9100h .Rezultatul se va plasa la locația adresei de memorie 9280h.

Problema se rezolva prin compararea primului element al șirului cu celelalte pe rând .

Daca prin comparare se găsește un număr mai mic decât precedentul atunci se renunță la acesta comparația continuând cu numărul mai mic.

Organigrama :



```

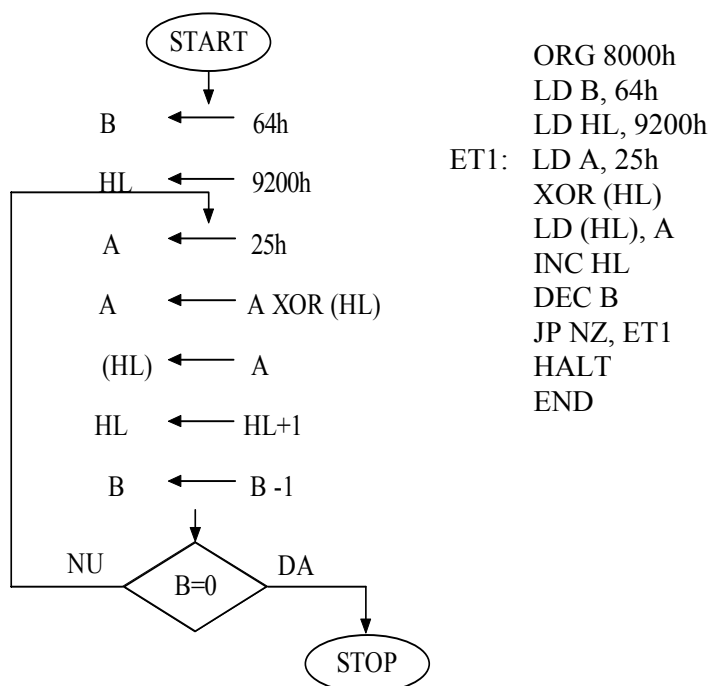
ORG 4000h
LD B, 32h
LD HL, 9100h
LD A, (HL)
ET1: INC HL
LD D, (HL)
CP D
JP C, ET2
LD A, D
ET2: DEC B
JP NZ, ET1
LD (9280h), A
HALT
END
  
```

Îndrumar de laborator 4

5. Să se realizeze un program în care se decodifică un șir de 100 de octeți codat după formula $M = N \text{ XOR } 25h$ (XOR =SAU exclusiv). Șirul se află poziționat începând de la adresa 9200h.

Pentru rezolvarea problemei se aplică șirului din nou operația sau exclusiv cu numărul 25h deoarece $(op1 \text{ XOR } 25h) \text{ XOR } 25h = op1$.

Se folosește registrul B care contorizează trecerea prin toate elementele șirului, iar în registrul A se introduce constanta 25h. Se aplică operația XOR pe rând fiecărui element după care se recopiază pe poziția sa înapoi în șir.



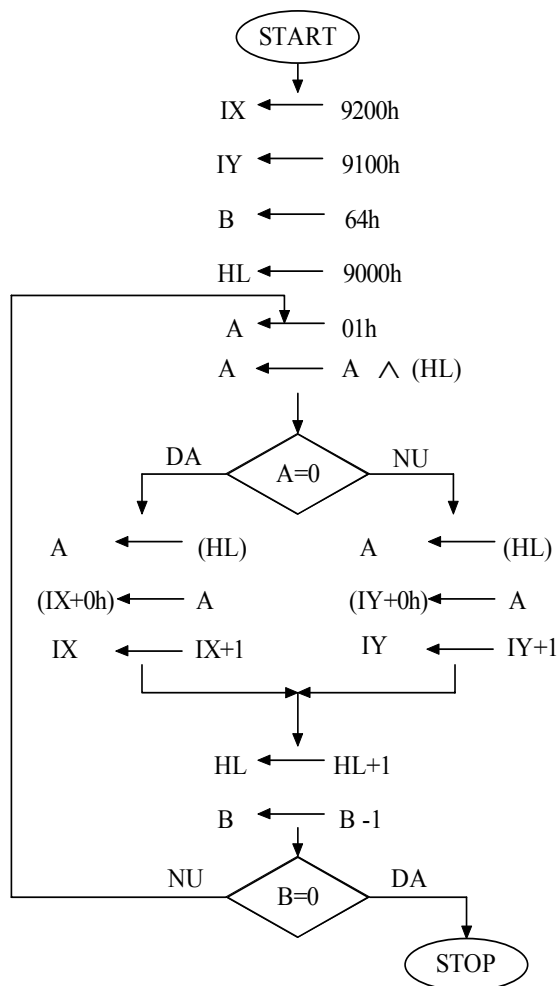
7. Se d un sir de 100 de octeți situat in memorie de la adresa 9000h .Sa se împartă șirul în două subșiruri unul cu numerele impare de tip zecimal iar celalalt cu numerele pare de același tip. Primul sir va fi plasat in memorie începând cu adresa 9100h iar al doilea începând cu 9200h .

Soluție :

Pentru rezolvare este necesara separarea numerelor pare de tip zecimal de cele impare ținând cont de condiția ca un număr e par in zecimal daca ultimul bit al sau este "0"logic ,iar la cele impare este "1"logic .

Folosind operația logica "si"(and) intre valoarea 01h si fiecare număr din sir in funcție de valoarea rezultatului se obțin cele doua subșiruri .Se știe ca $op1 \wedge 00h = 00h$ si $op1 \wedge 0FFh = op1$.

Organigrama :



```

ORG 5000h
LD IX, 9200h
LD IY, 9100h
LD B, 64h
LD HL, 9000h
ET1: LD A, 01h
      AND (HL)
      JP NZ, ET2
      LD A, (HL)
      LD (IX+0h), A
      INC IX
      JP ET3
ET2: LD A, (HL)
      LD (IY+0h), A
      INC IY
ET3: INC HL
      DEC B
      JP NZ, ET1
      HALT
      END
    
```

4. DESFĂȘURAREA LUCRĂRII

1. Se vor citi și prelua în scris toate informațiile teoretice prezentate până în acest moment. Se atrage atenția că aceste informații vor fi folosite și la desfășurarea altor lucrări de laborator.
2. Se vor studia problemele rezolvate, încercând găsirea altor posibilități de rezolvare.
3. Programele obiect ale problemelor rezolvate vor fi simulate în mediul de dezvoltare IAR și apoi introduse în memoria microsistemelor existente în laborator (prin intermediul programului monitor) și se vor rula. Se va observa efectul rulării și buna lor execuție.

Îndrumar de laborator 4

4. Se vor rezolva următoarele probleme propuse și se va urmări execuția corectă prin introducerea programelor obiect în memoria microsistemelor și rulare:

a) Realizați un program care va inițializa conținutul locațiilor de memorie cu adresele cuprinse între 9300h și 9350h cu numere începând de la 00h în ordine crescătoare.

b) Realizați un program care va inițializa conținutul locațiilor de memorie cu adresele cuprinse între A100h și A150h cu numere pare binar în ordine crescătoare începând cu constanta 24h

c) Să se realizeze un program în care să se ordoneze descrescător un șir de 100 de octeți situat începând de la adresa 8500h folosind metoda bulelor de aer .

d) Să se rezolve problema de mai sus prin aflarea minimelor sau a maximelor.

e) Să se realizeze un program în care se depune în memorie, de la adresa 9400h, constantele hexazecimale pare, reprezentate pe un octet, în ordine crescătoare, începând cu 00h.

f) Să se realizeze un program în care se depune în memorie, de la adresa 8500h, constantele hexazecimale impare, reprezentate pe un octet, în ordine descrescătoare, începând cu 0FFh.

g). Să se depună în memorie, de la adresa 800h, rezultatul operației SAU EXCLUSIV între locațiile corespunzătoare celor două zone de memorie folosite la problemele anterioare. (Astfel, la locația 8600h se va depune rezultatul operației SAU EXCLUSIV între conținutul locațiilor 8400h și 8500h și așa mai departe).

h). Se consideră un șir format din 150 numere binare reprezentate pe un octet. Șirul se află plasat în memoria unui microsistem de la adresa 800h. Să se realizeze un program care desparte termenii acestui șir, formând două subșiruri ce se vor scrie în memorie de la adresele 900h și 0a00h. În primul subșir vor fi trecute conținuturile locațiilor de la adresele pare din primul șir, iar în al doilea subșir vor fi scrise conținuturile locațiilor aflate la adresele impare din primul șir.

i). Să se realizeze un program de aflare a maximumului dintr-un șir de 50 de octeți situat în memorie începând de la adresa 100h .Rezultatul se va plasa la adresa 130h .Să se afle apoi câte numere din șir sunt egale cu maximumul.

j). Se da un șir de 100 de octeți situat în memorie de la adresa 2000h .Să se realizeze un program care împarte șirul în două subșiruri: unul cu numerele impare de tip binar iar celălalt cu numerele pare de același tip. Primul șir va fi plasat în memorie începând cu adresa 2100h iar al doilea începând cu 2200h .

k).La același șir ca la punctul anterior să se afle cel mai mic (sau mare) număr par zecimal și apoi binar.

Îndrumar de laborator 4